```
  object Status {
    def empty(marketId: String) =
      Status(marketId, Fixture("", "", ""), Odds(-1, -1, -1), 0)
  }
  ...
}
```

The `apply` method of the market is in essence the same as the wallet, with a slightly different initial state:

```
object Market {
  ...

 def apply(marketId: String): Behavior[Command] =
    EventSourcedBehavior[Command, Event, State](
      PersistenceId(typeKey.name, marketId),
      UninitializedState(Status.empty(marketId)),    ❶
      commandHandler = handleCommands,
      eventHandler = handleEvents)
      ...
  ...
}
```

❶ Initial state

The market is a finite state machine. For example, in an `UninitializedState`, the actor responds only to `Open` messages that cause it to switch to an `OpenState`. When it is in the `OpenState` state, receiving `Open` again is not defined as a valid entry and as such is interpreted as an invalid command. The `handleCommands` method expresses the entity's business requirements.

To express how the market reacts to each command depending on its state, there is no more concise way than the following code:

```
object Market {
  ...
```